Inheritance in .Net & C#

Inheritance is a fundamental concept in object-oriented programming that allows us to define a new class based on an existing class. The new class inherits the properties and methods of the existing class and can also add new properties and methods of its own. Inheritance promotes code reuse, simplifies code maintenance, and improves code organization.

It allows you to define a child class that reuses (inherits), extends, or modifies the behavior of a parent class. The class whose members are inherited is called the *base class*. The class that inherits the members of the base class is called the *derived class*.

In C#, there are several types of inheritance:

In C#, there are 4 types of inheritance:

1. **Single inheritance**: A derived class that inherits from only one base class.
2. **Multi-level inheritance**: A derived class that inherits from a base class and the derived class itself becomes the base class for another derived class.
3. **Hierarchical inheritance**: A base class that serves as a parent class for two or more derived classes.
4. **Multiple inheritance**: A derived class that inherits from two or more base classes.

C# and .NET support *single inheritance* only. That is, a class can only inherit from a single class. However, inheritance is transitive, which allows you to define an inheritance hierarchy for a set of types. In other words, type D can inherit from type C, which inherits from type B, which inherits from the base class type A. Because inheritance is transitive, the members of type A are available to type D.

Not all members of a base class are inherited by derived classes. The following members are not inherited:

- Static constructors, which initialize the static data of a class.
- Instance constructors, which you call to create a new instance of the class. Each class must define its own constructors.
- Finalizers, which are called by the runtime's garbage collector to destroy instances of a class.

While all other members of a base class are inherited by derived classes, whether they are visible or not depends on their accessibility. A member's accessibility affects its visibility for derived classes as follows:

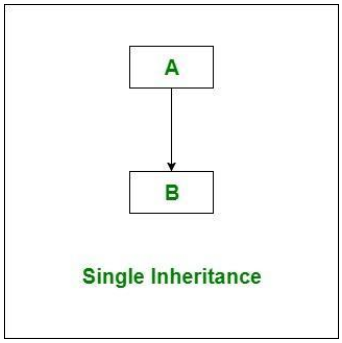
it is possible to inherit fields and methods from one class to another. We group the "inheritance concept" into two categories:

- **Derived Class** (child) - the class that inherits from another class
- **Base Class** (parent) - the class being inherited from

Types of Inheritance in C#

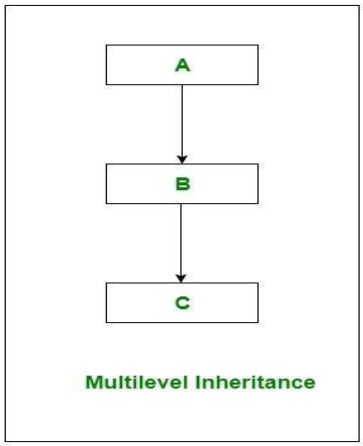
Below are the different types of inheritance which is supported by C# in different combinations.

1. **Single Inheritance:** In single inheritance, subclasses inherit the features of one superclass. In image below, the class A serves as a base class for the derived class B.

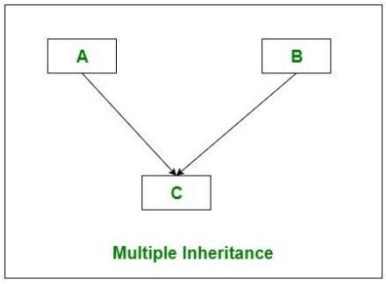


2. **Multilevel Inheritance:** In Multilevel Inheritance, a derived class will be inheriting a base class and as well as the derived class also act as the base class to other class. In below image, class A serves as a base class for the

derived class B, which in turn serves as a base class for the derived class C.

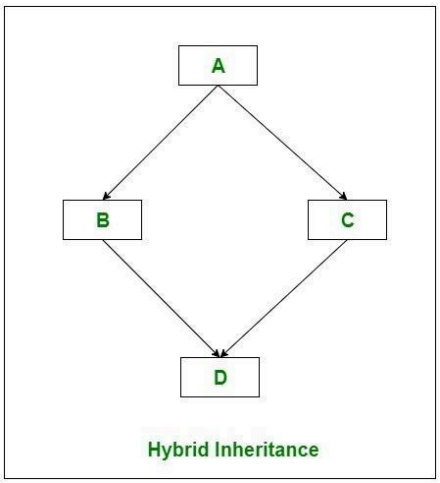


3. **Hierarchical Inheritance:** In Hierarchical Inheritance, one class serves as a superclass (base class) for more than one subclass. In below image, class A serves as a base class for the derived class B, C, and D.
4. **Multiple Inheritance(Through Interfaces):**In Multiple inheritance, one class can have more than one superclass and inherit features from all parent classes. Please note that **C# does not support multiple inheritance** with classes. In C#, we can achieve multiple inheritance only through Interfaces. In the image below, Class C is derived from interface A and B.



5. **Hybrid Inheritance(Through Interfaces):** It is a mix of two or more of the above types of inheritance. Since C# doesn't support multiple inheritance with classes, the hybrid inheritance is also not possible with classes. In C#,

we can achieve hybrid inheritance only through Interfaces.



Important facts about inheritance in C#

- **Default Superclass**: Except Object class, which has no superclass, every class has one and only one direct superclass(single inheritance). In the absence of any other explicit superclass, every class is implicitly a subclass of Object class.
- **Superclass can only be one:** A superclass can have any number of subclasses. But a subclass can have only **one** superclass. This is because C# does not support multiple inheritance with classes. Although with interfaces, multiple inheritance is supported by C#.
- **Inheriting Constructors:** A subclass inherits all the members (fields, methods) from its superclass. Constructors are not members, so they are not inherited by subclasses, but the constructor of the superclass can be invoked from the subclass.
- **Private member inheritance:** A subclass does not inherit the private members of its parent class. However, if the superclass has properties(get and set methods) for accessing its private fields, then a subclass can inherit.

To inherit from a class, use the : symbol.

In the example below, the Car class (child) inherits the fields and methods from the Vehicle class (parent):

EXAMPLE

```
class A
{
public void Method1()
{
Console.WriteLine("Method 1");
}
public void Method2()
{
Console.WriteLine("Method 2");
}
}
```

Here, we have created class A with two public methods, i.e., Method1 and Method2. Now, I want the same two methods in another class, i.e., class B. One way to do this is to copy the above two methods and paste them into class B as follows.

```
class B
{
public void Method1()
{
Console.WriteLine("Method 1");
}
public void Method2()
{
```

```
Console.WriteLine("Method 2");
}
}
```

If we do this, then it is not code re-usability. It is code rewriting that affects the size of the application. So, without rewriting, what we need to do is, we need to perform inheritance here as follows. Here, class B is inherited from class A, and hence, inside the Main method, we create an instance of class B and invoke the methods which are defined in Class A.

```
class B : A
{
static void Main()
{
B obj = new B();
obj.Method1();
obj.Method2();
}
}
```

Once you perform the Inheritance, class B can take the two members defined in class A. Why? Because all the properties of a Parent belong to Children. Here, class A is the Parent/Super/Base class, and Class B is the Child/Sub/Derived class.

Let us understand one more thing. Please observe the following image when we say obj. you can see the intelligence it is showing the two Methods, i.e., Method1 and Method2. So, the child class can consume the members of the parent class as if it is the owner. Now, if you see the description of either Method1 or Method2, it is showing void A.Method1() and void A.Method2(). That means Method1 or Method2 belongs to class A only. But class B can consume the member as if it is the owner. See, I can drive my father's car as if I am the owner, but still, the registration name is my father. Similarly, class B can call the methods as the method is its own, but internally, the methods belong to Class A.

The complete code example is given below. In the below example, class A defined two members, and class B is inherited from Class A. In class B, within the Main method, we created an instance of class B and called the two methods.

```
using System;
namespace InheritanceDemo
{
class A
{
public void Method1()
{
Console.WriteLine("Method 1");
}
public void Method2()
{
Console.WriteLine("Method 2");
}
}
class B : A
{
static void Main()
{
B obj = new B();
obj.Method1();
obj.Method2();
```

```
Console.ReadKey();
}
}
}
```

Output:

Method 1
Method 2

Now, let us add a new method, i.e., Method3 in Class B, as follows. Inside the Main method, if you see the method description, it shows that the method belongs to class B.

The complete example is given below.

```
using System;
namespace InheritanceDemo
{
class A
{
public void Method1()
{
Console.WriteLine("Method 1");
}
public void Method2()
{
```

```
Console.WriteLine("Method 2");
}
}
class B : A
{
public void Method3()
{
Console.WriteLine("Method 3");
}
static void Main()
{
B obj = new B();
obj.Method1();
obj.Method2();
obj.Method3();
Console.ReadKey();
}
}
}
```

Output:

Method 1
Method 2
Method 3

How many methods are there in class B?

Now, you may have one question: how many methods are there in class B? The answer is 3. See, all the properties that my father has given to me, plus all the properties that I am earning, are my property only. So, what is my property means I don't say what I earned; I also say what my father has given to me. So, in the same way, how many methods are there in class B means 3 methods. Two methods were inherited from the parent class A plus one method which we defined in class B. So, we can say Class A contains two methods, and Class B contains 3 methods.

This is the simple process of Inheritance in C#. Simply put a colon (:) between the Parent and Child class. But when you are working with Inheritance, 6 things or rules are required to learn and remember. Let us learn those 6 important Rules one by one.

Rule1:

In C#, the parent classes constructor must be accessible to the child class; otherwise, the inheritance would not be possible because when we create the child class object, first it goes and calls the parent class constructor so that the parent class variable will be initialized and we can consume them under the child class.

Right now, in our example, both class A and class B have implicit constructors. Yes, every class in C# contains an implicit constructor if as a developer we did not define any constructor explicitly. We already learned it in our constructor section.

If a constructor is defined implicitly, then it is a public constructor. In our example, class B can access the class A implicit constructor as it is public. Now, let us define one explicit constructor in Class A as follows.

```
class A
{
public A()
{
Console.WriteLine("Class A Constructor is Called");
}
public void Method1()
{
Console.WriteLine("Method 1");
}
public void Method2()
```

```
{
Console.WriteLine("Method 2");
}
}
```

With the above changes in place, if you run the application code, you will get the following

Output:

Class A Constructor is Called
Method 1
Method 2
Method 3

.

When you execute the code, the class A constructor is first called, and that is what you can see in the output. Why? This is because whenever the child class instance is created, the child class constructor will implicitly call its parent class constructors. This is a rule.

Right now, the child class contains an implicit constructor, and that implicit constructor calls the parent class constructor. But the Parent class A constructor is not implicitly. It is explicitly now, and inside that parent class constructor, we have written print statement and print statement printing some message on the console window.

Example-1

```
class Vehicle  // base class (parent)
{
  public string brand = "Ford";  // Vehicle field
  public void honk()          // Vehicle method
  {
    Console.WriteLine("Tuut, tuut!");
  }
```

```
}

class Car : Vehicle  // derived class (child)

{

  public string modelName = "Mustang";  // Car field

}

class Program

{

  static void Main(string[] args)

  {

    // Create a myCar object

    Car myCar = new Car();


 // Call the honk() method (From the Vehicle class) on the myCar object

    myCar.honk();

    // Display the value of the brand field (from the Vehicle class) and the value of the modelName from the Car class

    Console.WriteLine(myCar.brand + " " + myCar.modelName);

  }

}
```

OUTPUT

```
Tuut, tuut!
Ford Mustang
```

EXAMPLE-2

C# Single Level Inheritance Example: Inheriting Fields

When one class inherits another class, it is known as single level inheritance. Let's see the example of single level inheritance which inherits the fields only.

```
using System;
public class Employee
 {
   public float salary = 40000;
 }
public class Programmer: Employee
{  780[
class TestInheritance{
   public static void Main(string[] args)
    {
       Programmer p1 = new Programmer();

       Console.WriteLine("Salary: " + p1.salary);
       Console.WriteLine("Bonus: " + p1.bonus);

    }
 }
```

Output:

```
Salary: 40000
Bonus: 10000
```

In the above example, Employee is the **base** class and Programmer is the **derived** class.

C# Single Level Inheritance Example: Inheriting Methods

Let's see another example of inheritance in C# which inherits methods only.

```
using System;
  public class Animal
```

```
3.     {
4.         public void eat()
5.  {
6.  Console.WriteLine("Eating...");
7.   }
8.    }
9.    public class Dog: Animal
10.   {
11.       public void bark()
12. {
13. Console.WriteLine("Barking...");
14. }
15.   }
16.   class TestInheritance2{
17.       public static void Main(string[] args)
18.        {
19.           Dog d1 = new Dog();
20.           d1.eat();
21.           d1.bark();
22.        }
23.    }
```

Output:

```
Eating...
Barking...
```

C# Multi Level Inheritance Example

When one class inherits another class which is further inherited by another class, it is known as multi level inheritance in C#. Inheritance is transitive so the last derived class acquires all the members of all its base classes.

Let's see the example of multi level inheritance in C#.

```
1.  using System;
2.    public class Animal
3.    {
4.        public void eat() { Console.WriteLine("Eating..."); }
```

```
5.    }
6.    public class Dog: Animal
7.    {
8.       public void bark() { Console.WriteLine("Barking..."); }
9.    }
10.   public class BabyDog : Dog
11.   {
12.      public void weep() { Console.WriteLine("Weeping..."); }
13.   }
14.   class TestInheritance2{
15.      public static void Main(string[] args)
16.       {
17.          BabyDog d1 = new BabyDog();
18.          d1.eat();
19.          d1.bark();
20.          d1.weep();
21.       }
22.    }
```

Output:

```
Eating...
Barking...
Weeping...
```

EXAMPLE-3 - ALL INHERITANCE

using System;

// single inheritance

```
class Animal {
   public void Eat() {
```

```
        Console.WriteLine("Animal is eating.");
    }
}

class Dog : Animal {
    public void Bark() {
        Console.WriteLine("Dog is barking.");
    }
}

// multi-level inheritance
class Mammal : Animal {
    public void Run() {
        Console.WriteLine("Mammal is running.");
    }
}

class Horse : Mammal {
    public void Gallop() {
        Console.WriteLine("Horse is galloping.");
    }
}

// hierarchical inheritance
class Bird : Animal {
    public void Fly() {
        Console.WriteLine("Bird is flying.");
    }
}

class Eagle : Bird {
    public void Hunt() {
        Console.WriteLine("Eagle is hunting.");
    }
}

class Penguin : Bird {
    public void Swim() {
        Console.WriteLine("Penguin is swimming.");
    }
```

```
}

// multiple inheritance
interface I1 {
   void Method1();
}

interface I2 {
   void Method2();
}

class MyClass : I1, I2 {
   public void Method1() {
      Console.WriteLine("Method1 is called.");
   }

   public void Method2() {
      Console.WriteLine("Method2 is called.");
   }
}

// main program
class Program {
   static void Main(string[] args) {
      // single inheritance
      Dog dog = new Dog();
      dog.Eat();
      dog.Bark();

      // multi-level inheritance
      Horse horse = new Horse();
      horse.Eat();
      horse.Run();
      horse.Gallop();

      // hierarchical inheritance
      Eagle eagle = new Eagle();
      Penguin penguin = new Penguin();
      eagle.Fly();
```

```
        eagle.Hunt();
        penguin.Fly();
        penguin.Swim();

        // multiple inheritance
        MyClass myClass = new MyClass();
        myClass.Method1();
        myClass.Method2();

        Console.ReadLine();
    }
}
```

Output

Animal is eating.

Dog is barking.

Animal is eating.

Mammal is running.

Horse is galloping.

Bird is flying.

Eagle is hunting.

Bird is flying.

Penguin is swimming.

Method1 is called.

Method2 is called.

Advantages of Inheritance:

1. Code Reusability: Inheritance allows us to reuse existing code by inheriting properties and methods from an existing class.

2. Code Maintenance: Inheritance makes code maintenance easier by allowing us to modify the base class and have the changes automatically reflected in the derived classes.
3. Code Organization: Inheritance improves code organization by grouping related classes together in a hierarchical structure.

Disadvantages of Inheritance:

1. Tight Coupling: Inheritance creates a tight coupling between the base class and the derived class, which can make the code more difficult to maintain.
2. Complexity: Inheritance can increase the complexity of the code by introducing additional levels of abstraction.
3. Fragility: Inheritance can make the code more fragile by creating dependencies between the base class and the derived class.

POLYMORPHISM

The term "Polymorphism" is the combination of "poly" + "morphs" which means many forms. It is a greek word. In object-oriented programming, we use 3 main concepts: inheritance, encapsulation and polymorphism.

There are two types of polymorphism in C#: compile time polymorphism and runtime polymorphism. Compile time polymorphism is achieved by method overloading and operator overloading in C#. It is also known as static binding or early binding. Runtime polymorphism in achieved by method overriding which is also known as dynamic binding or late binding.

Polymorphism in C#

Polymorphism is one of the fundamental OOP concepts and is a term used to describe situations where something takes various roles or forms. In the programming world, these things can be operators or functions.

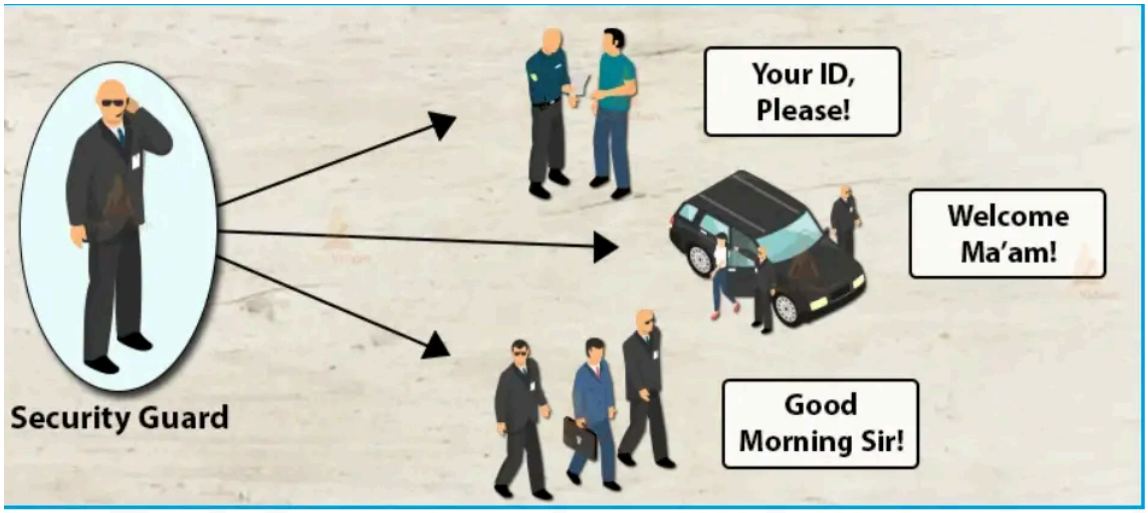
The word polymorphism is derived from two Greek words: poly and morphs. The word “Poly” means many, and “morphs” means forms. Therefore, polymorphism means “many forms” or we can say that the word polymorphism means the ability to take more than one form. That is one thing that can take many forms.

Example1:

Suppose you are in a classroom, then at that time, you will behave like a student. But when you are in the shopping mall, at that time you will behave like a customer. Again, when you are traveling on a bus, then you will behave like a passenger. And when you are at your home at that time, you will behave like a son or daughter. Here, you are one person but performing different behaviors. This is nothing but polymorphism. The behaviors change based on the place.

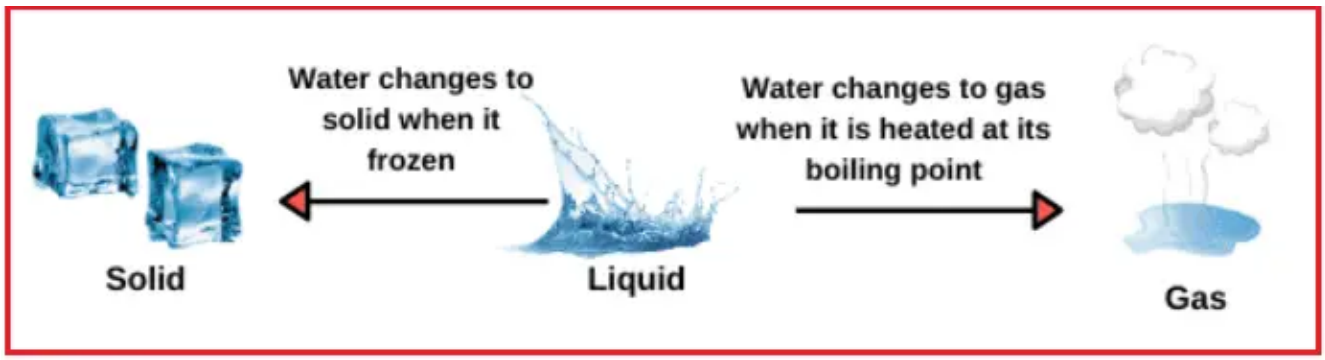
Example2:

A security guard in an organization behaves differently with different people entering the organization. The security behaves in a different way when the Boss comes and, in another way, when the employees come. And when the customers enter, the same security guard will respond differently. So here, the behavior of the security guard changes from person to person. It depends on the member who is entering the organization.



Example3:

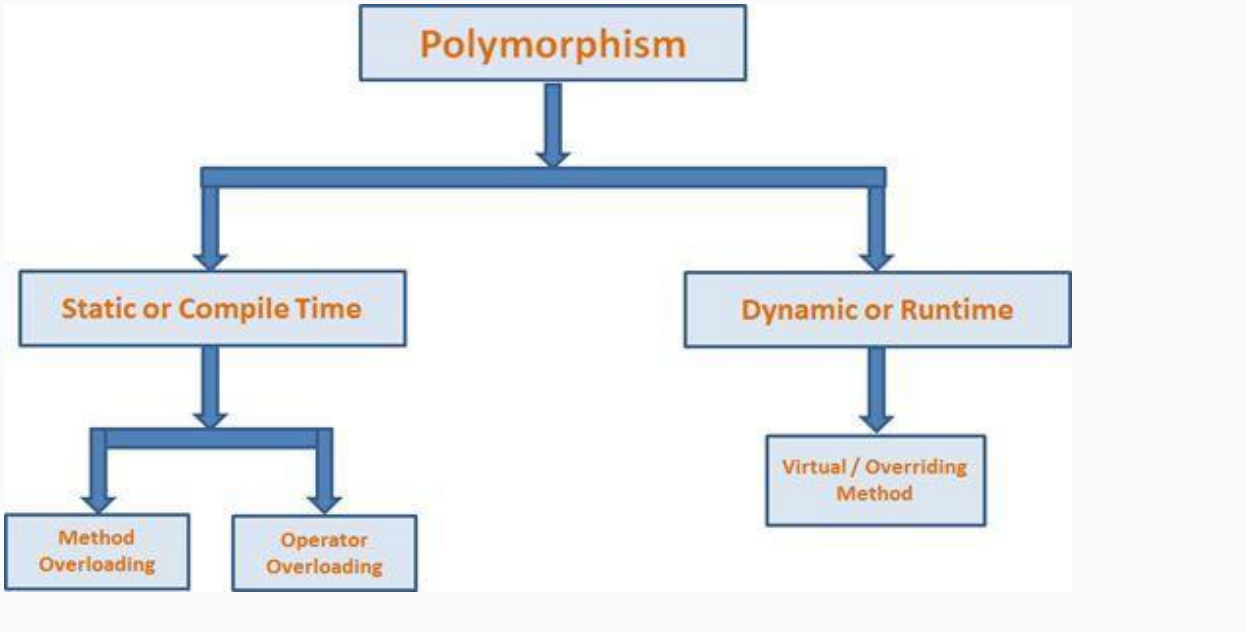
Another good real-time example of polymorphism is water. We all know that water is a liquid at normal temperature, but it changes to a solid when it is frozen, and the same water changes to a gas when it is heated at its boiling point. This is also polymorphism.



Types of Polymorphism

After getting the basic idea of polymorphism, let's learn the types of polymorphism in C#.

There are two types of polymorphism:



1. **Static Polymorphism / Compile-Time Polymorphism / Early Binding**
2. **Dynamic Polymorphism / Run-Time Polymorphism / Late Binding**

1. Compile Time Polymorphism

In compile time polymorphism, the compiler identifies which method is being called at the compile time.

In C#, we achieve compile time polymorphism through 2 ways:

- Method overloading
- Operator overloading

```
using System;
namespace MethodOverloading
{
class Program
{
public void Add(int a, int b)
```

```
{
Console.WriteLine(a + b);
}
public void Add(float x, float y)
{
Console.WriteLine(x + y);
}
public void Add(string s1, string s2)
{
Console.WriteLine(s1 + " " + s2);
}
static void Main(string[] args)
{
Program obj = new Program();
obj.Add(10, 20);
obj.Add(10.5f, 20.5f);
obj.Add("Pranaya", "Rout");
Console.ReadKey();
}
}
}
```

Output:

Example of Polymorphism

```
using System;
class Program
{
    // method does not take any parameter
    public void greet()
    {
        Console.WriteLine("Hello");
    }


    // method takes one string parameter
    public void greet(string name)
    {
        Console.WriteLine("Hello " + name);
    }


    static void Main(string[] args)
    {
        Program p1 = new Program();

        // calls method without any argument
        p1.greet();

        //calls method with an argument
        p1.greet("Tim");

    }
}
```

Output

```
Hello
Hello Tim
```

In the above example, we have created a class `Program` inside which we have two methods of the same name `greet()`.
Here, one of the `greet()` methods takes no parameters and displays `"Hello"`.
While the other `greet()` method takes a parameter and displays `"Hello Tim"`.
Hence, the `greet()` method behaves differently in different scenarios. Or, we can say `greet()` is polymorphic.

C# Method Overloading

In a C# class, we can create methods with the same name in a class if they have:

- different numbers of parameter
- types of parameter

For example,

```
void totalSum() {...}
void totalSum(int a) {...}
void totalSum(int a, int b) {...}
void totalSum(float a, float b) {...}
```

Here we have different types and numbers of parameters in `totalSum()`. This is known as method overloading in C#. The same method will perform different operations based on the parameter.
Look at the example below,

```
using System;
class Program
{

    // method adds two integer numbers
    void totalSum(int a, int b)
```

```
    {
        Console.WriteLine("The sum of numbers is " + (a +
b));
    }

    // method adds two double-type numbers
    // totalSum() method is overloaded
    void totalSum(double a, double b)
    {
        Console.WriteLine("The sum of numbers is " + (a +
b));
    }

    static void Main(string[] args)
    {

        Program sum1 = new Program();
        sum1.totalSum(5, 7);
        sum1.totalSum(53.5, 8.7);
    }
}
```

Output

```
The sum of numbers is 12
The sum of numbers is 62.2
```

In the above example, the class `Program` contains a method named `totalSum()` that is overloaded. The `totalSum()` method prints:

- sum of integers if two integers are passed as an argument
- sum of doubles if two doubles are passed as an argument

Function Overloading example

You can have multiple definitions for the same function name in the same scope. The definition of the function must differ from each other by the types and/or the number of arguments in the argument list. You cannot overload function declarations that differ only by return type.

The following example shows using function **print()** to print different data types –

```
using System;

namespace PolymorphismApplication {
   class Printdata {
      void print(int i) {
         Console.WriteLine("Printing int: {0}", i );
      }
      void print(double f) {
         Console.WriteLine("Printing float: {0}" , f);
      }
      void print(string s) {
         Console.WriteLine("Printing string: {0}", s);
      }
      static void Main(string[] args) {
         Printdata p = new Printdata();

         // Call print to print integer
         p.print(5);

         // Call print to print float
         p.print(500.263);

         // Call print to print string
         p.print("Hello C++");
         Console.ReadKey();
      }
```

```
    }
}
```

OUTPUT

Printing int: 5
Printing float: 500.263
Printing string: Hello C++

Function or Method Overloading

You can have multiple definitions for the same function name in the same scope. The definition of the function must differ from each other by the types and/or the number of arguments in the argument list. You cannot overload function declarations that differ only by return type.

The following example shows using function **print()** to print different data types −

```
using System;

namespace PolymorphismApplication {
  class Printdata {
    void print(int i) {
      Console.WriteLine("Printing int: {0}", i );
    }
    void print(double f) {
      Console.WriteLine("Printing float: {0}" , f);
    }
    void print(string s) {
      Console.WriteLine("Printing string: {0}", s);
    }
    static void Main(string[] args) {
      Printdata p = new Printdata();

      // Call print to print integer
```

```
        p.print(5);

        // Call print to print float
        p.print(500.263);

        // Call print to print string
        p.print("Hello C++");
        Console.ReadKey();
    }
  }
}
```

OUTPUT

Printing int: 5
Printing float: 500.263
Printing string: Hello C++

C# Operator Overloading

Some operators in C# behave differently with different operands. For example,

- + operator is overloaded to perform numeric addition as well as string concatenation and

Now let's see how we can achieve polymorphism using operator overloading.

The + operator is used to add two entities. However, in C#, the + operator performs two operations:
1. Adding two numbers,

```
int x = 7;
int y = 5;
int sum = x + y;
Console.WriteLine(sum);
// Output: 12
```

2. Concatenating two strings,

```
string firstString = "Harry";
string secondString = "Styles";

string concatenatedString = firstString + secondString;
Console.WriteLine(concatenatedString);
// Output: HarryStyles
```

Here, we can see that the + operator is overloaded in C# to perform two operations: addition and concatenation.

---

2. Runtime Polymorphism

In runtime polymorphism, the method that is called is determined at the runtime not at compile time.

The runtime polymorphism is achieved by:

- Method Overriding

Let's discuss method overriding in detail.

Dynamic Polymorphism

C# allows you to create abstract classes that are used to provide partial class implementation of an interface. Implementation is completed when a derived class inherits from it. **Abstract** classes contain abstract methods, which are implemented by the derived class. The derived classes have more specialized functionality.

Here are the rules about abstract classes –

- You cannot create an instance of an abstract class
- You cannot declare an abstract method outside an abstract class
- When a class is declared **sealed**, it cannot be inherited, abstract classes cannot be declared sealed.

Method Overriding in C#

During inheritance in C#, if the same method is present in both the superclass and the subclass. Then, the method in the subclass overrides the same method in the superclass. This is called method overriding.

In this case, the same method will perform one operation in the superclass and another operation in the subclass.

We can use `virtual` and `override` keywords to achieve method overriding. Let's see the example below,

```
using System;
namespace PolymorphismDemo
{
class Class1
{
//Virtual Function (Overridable Method)
public virtual void Show()
{
//Parent Class Logic Same for All Child Classes
Console.WriteLine("Parent Class Show Method");
}
}
class Class2 : Class1
```

```
{
//Overriding Method
public override void Show()
{
//Child Class Reimplementing the Logic
Console.WriteLine("Child Class Show Method");
}
}
class Program
{
static void Main(string[] args)
{
Class1 obj1 = new Class2();
obj1.Show(); //Resolve at Runtime
Console.ReadKey();
}
}
}
```

Output: Child Class Show Method

Example------

```
using System;
class Polygon
{
    // method to render a shape
    public virtual void render()
    {
        Console.WriteLine("Rendering Polygon...");
    }
```

```
}

class Square : Polygon
{
    // overriding render() method
    public override void render()
    {
        Console.WriteLine("Rendering Square...");
    }

}
class myProgram
{
    public static void Main()
    {
        // obj1 is the object of Polygon class
        Polygon obj1 = new Polygon();

        // calls render() method of Polygon Superclass
        obj1.render();

        // here, obj1 is the object of derived class Square
        obj1 = new Square();

        // calls render() method of derived class Square
        obj1.render();
    }
}
```

Output

```
Rendering Polygon...
Rendering Square...
```

In the above example, we have created a superclass: `Polygon` and a subclass: `Square`.

Notice, we have used `virtual` and `override` with methods of the base class and derived class respectively. Here,

- `virtual` - allows the method to be overridden by the derived class
- `override` - indicates the method is overriding the method from the base class

In this way, we achieve method overriding in C#.

Note: A method in derived class overrides the method in base class if the method in derived class has the **same name**, **same return type** and **same parameters** as that of the base class.

C# Runtime Polymorphism Example 2

Let's see a another example of runtime polymorphism in C# where we are having two derived classes.

```
1.  using System;
2.  public class Shape{
3.     public virtual void draw(){
4.        Console.WriteLine("drawing...");
5.     }
6.  }
7.  public class Rectangle: Shape
8.  {
9.     public override void draw()
10.    {
11.       Console.WriteLine("drawing rectangle...");
12.    }
13.
14. }
15. public class Circle : Shape
16. {
17.    public override void draw()
18.    {
19.       Console.WriteLine("drawing circle...");
20.    }
21.
22. }
23. public class TestPolymorphism
24. {
25.    public static void Main()
26.    {
27.       Shape s;
28.       s = new Shape();
29.       s.draw();
30.       s = new Rectangle();
31.       s.draw();
32.       s = new Circle();
33.       s.draw();
```

```
34.
35.     }
36. }
```

Output:

```
drawing...
drawing rectangle...
drawing circle...
```

Runtime Polymorphism with Data Members

Runtime Polymorphism can't be achieved by data members in C#. Let's see an example where we are accessing the field by reference variable which refers to the instance of derived class.

```
1.  using System;
2.  public class Animal{
3.     public string color = "white";
4.
5.  }
6.  public class Dog: Animal
7.  {
8.     public string color = "black";
9.  }
10. public class TestSealed
11. {
12.    public static void Main()
13.    {
14.       Animal d = new Dog();
15.       Console.WriteLine(d.color);
16.
17.    }
18. }
```

Output:

white

The following program demonstrates an abstract class −

```
using System;

namespace PolymorphismApplication {
   abstract class Shape {
```

```
   public abstract int area();
  }

  class Rectangle:  Shape {
    private int length;
    private int width;

    public Rectangle( int a = 0, int b = 0) {
      length = a;
      width = b;
    }
    public override int area () {
      Console.WriteLine("Rectangle class area :");
      return (width * length);
    }
  }
  class RectangleTester {
    static void Main(string[] args) {
      Rectangle r = new Rectangle(10, 7);
      double a = r.area();
      Console.WriteLine("Area: {0}",a);
      Console.ReadKey();
    }
  }
}
```

When the above code is compiled and executed, it produces the following result –

Rectangle class area :
Area: 70

When you have a function defined in a class that you want to be implemented in an inherited class(es), you use **virtual** functions. The

virtual functions could be implemented differently in different inherited class and the call to these functions will be decided at runtime.

Dynamic polymorphism is implemented by **abstract classes** and **virtual functions**.

The following program demonstrates this −

```
using System;

namespace PolymorphismApplication {
  class Shape {
    protected int width, height;

    public Shape( int a = 0, int b = 0) {
      width = a;
      height = b;
    }
    public virtual int area() {
      Console.WriteLine("Parent class area :");
      return 0;
    }
  }
  class Rectangle: Shape {
    public Rectangle( int a = 0, int b = 0): base(a, b) {

    }
    public override int area () {
      Console.WriteLine("Rectangle class area :");
      return (width * height);
    }
  }
  class Triangle: Shape {
    public Triangle(int a = 0, int b = 0): base(a, b) {
    }
```

```
    public override int area() {
      Console.WriteLine("Triangle class area :");
      return (width * height / 2);
    }
  }
  class Caller {
    public void CallArea(Shape sh) {
      int a;
      a = sh.area();
      Console.WriteLine("Area: {0}", a);
    }
  }
  class Tester {
    static void Main(string[] args) {
      Caller c = new Caller();
      Rectangle r = new Rectangle(10, 7);
      Triangle t = new Triangle(10, 5);

      c.CallArea(r);
      c.CallArea(t);
      Console.ReadKey();
    }
  }
}
```

When the above code is compiled and executed, it produces the following result –

Rectangle class area:
Area: 70
Triangle class area:
Area: 25

Why Polymorphism?

Polymorphism allows us to create consistent code. In the previous example, we can also create a different method: `renderSquare()` to render Square.

This will work perfectly. However, for every shape, we need to create different methods. It will make our code inconsistent.

To solve this, polymorphism in C# allows us to create a single method `render()` that will behave differently for different shapes.

Summary

The meaning of polymorphism is one name has multiple forms.

The following are the two types of polymorphism:

1. Static or compile-time polymorphism (method overloading and operator overloading).
2. Dynamic or runtime polymorphism (for example, overriding).

Other points about polymorphism:

- Method Overriding differs from shadowing.
- Using the "new" keyword, we can hide the base class member."
- We can prevent a derived class from overriding virtual members.
- We can access a base class virtual member from the derived class.

Example- C# Runtime Polymorphism

Let's see a simple example of runtime polymorphism in C#.

```
1. using System;
2. public class Animal{
3.     public virtual void eat(){
4.         Console.WriteLine("eating...");
5.     }
6. }
7. public class Dog: Animal
8. {
9.     public override void eat()
10.    {
11.        Console.WriteLine("eating bread...");
12.    }
```

```
13.
14. }
15. public class TestPolymorphism
16. {
17.     public static void Main()
18.     {
19.         Animal a= new Dog();
20.         a.eat();
21.     }
22. }
```

Output:

```
eating bread...
```

Example 2- C# Runtime Polymorphism

Let's see a another example of runtime polymorphism in C# where we are having two derived classes.

```
1.  using System;
2.  public class Shape{
3.      public virtual void draw(){
4.          Console.WriteLine("drawing...");
5.      }
6.  }
7.  public class Rectangle: Shape
8.  {
9.      public override void draw()
10.     {
11.         Console.WriteLine("drawing rectangle...");
12.     }
13.
14. }
15. public class Circle : Shape
```

```
{
    public override void draw()
    {
        Console.WriteLine("drawing circle...");
    }

}
public class TestPolymorphism
{
    public static void Main()
    {
        Shape s;
        s = new Shape();
        s.draw();
        s = new Rectangle();
        s.draw();
        s = new Circle();
        s.draw();

    }
}
```

Output:

```
drawing...
drawing rectangle...
drawing circle...
```

Runtime Polymorphism with Data Members

Runtime Polymorphism can't be achieved by data members in C#. Let's see an example where we are accessing the field by reference variable which refers to the instance of derived class.

```
using System;
public class Animal{
    public string color = "white";

```

```
}
public class Dog: Animal
{
    public string color = "black";
}
public class TestSealed
{
    public static void Main()
    {
        Animal d = new Dog();
        Console.WriteLine(d.color);

    }
}
```

Output:

```
white
```

Difference between Inheritance and Polymorphism:

S.NO	Inheritance	Polymorphism
1.	Inheritance is one in which a new class is created (derived class) that inherits the features from the already existing class(Base class).	Whereas polymorphism is that which can be defined in multiple forms.

S.NO	Inheritance	Polymorphism
2.	It is basically applied to classes.	Whereas it is basically applied to functions or methods.
3.	Inheritance supports the concept of reusability and reduces code length in object-oriented programming.	Polymorphism allows the object to decide which form of the function to implement at compile-time (overloading) as well as run-time (overriding).
4.	Inheritance can be single, hybrid, multiple, hierarchical and multilevel inheritance.	Whereas it can be compiled-time polymorphism (overload) as well as run-time polymorphism (overriding).
5.	It is used in pattern designing.	While it is also used in pattern designing.
6.	**Example :** The class bike can be inherit from the class of two-wheel vehicles, which is turn could be a subclass of vehicles.	**Example :** The class bike can have method name set_color(), which changes the bike's color based on the name of

S.NO	Inheritance	Polymorphism
		color you have entered.

Visibility In C#

The visibility of a class, a method, a variable or a property tells us how this item can be accessed. The most common types of visibility are private and public, but there are actually several other types of visibility within C#. Here is a complete list, and although some of them might not feel that relevant to you right now, you can always come back to this page and read up on them:

public - the member can be reached from anywhere. This is the least restrictive visibility. Enums and interfaces are, by default, publicly visible.

protected - members can only be reached from within the same class, or from a class which inherits from this class.

internal - members can be reached from within the same project only.

protected internal - the same as internal, except that classes which inherit from this class can reach its members; even from another project.

private - can only be reached by members from the same class. This is the most restrictive visibility. Classes and structs are by default set to private visibility.

So for instance, if you have two classes: Class1 and Class2, private members from Class1 can only be used within Class1. You can't create a new instance of Class1 inside of Class2, and then expect to be able to use its private members.

If Class2 inherits from Class1, then only non-private members can be reached from inside of Class2.

Controls visibility in C#

To toggle the visibility of a control, I set a panel control visibility in the code behind. The visibility of another panel, panel visibility , is set to false if the values of Username and UserId match.

```
if (UserName == UserID))

{

    pnl_linkbuttons.Visible = false;

}
```

Is it possible to set control visibility in C# from the code behind using a different approach?

Solution 1:

```
pnl_linkbuttons.Visible = UserName == UserID;
```

Solution 2:

When it comes to asp.net, jQuery or javascript are viable options, while C# is the go-to language for windows forms development.

In case you were using Silverlight or WPF, it is possible to establish visibility through XAML.

If you're working with XAML, refer to this link for guidance on collapsing a XAML TextBlock element that has no data.

You can find information about the System.Windows.Trigger class on the MSDN website at http://msdn.microsoft.com/en-us/library/system.windows.trigger.aspx.

C# - How do I determine visibility of a control?, I have a TabControl that contains several tabs. Each tab has one UserControl on it. I would like to check the visibility of a control x on UserControl A from UserControl B.I figured that doing x.Visible from UserControl B would be good enough. As it turns out, it was displaying false in the debugger even though I set it explicitly to

How to toggle visibility in Windows Forms C#

I frequently change the visibility of a few labels, and when I do so, I simply utilize.

```
Label1.Visible = true;

Label2.Visible = true;

Label3.Visible = true;
```

or

```
Label1.Visible = false;

Label2.Visible = false;

Label3.Visible = false;
```

In order to improve the readability of my code, I am considering creating a function. However, I require a function that can toggle visibility rather than simply turning it on or off. Can this be achieved?

Solution 1:

you mean simply invert:

```
void ToggleLabel(Label l)

{

   l.Visible = ! l.Visible ;

}
```

Method Overriding in C#

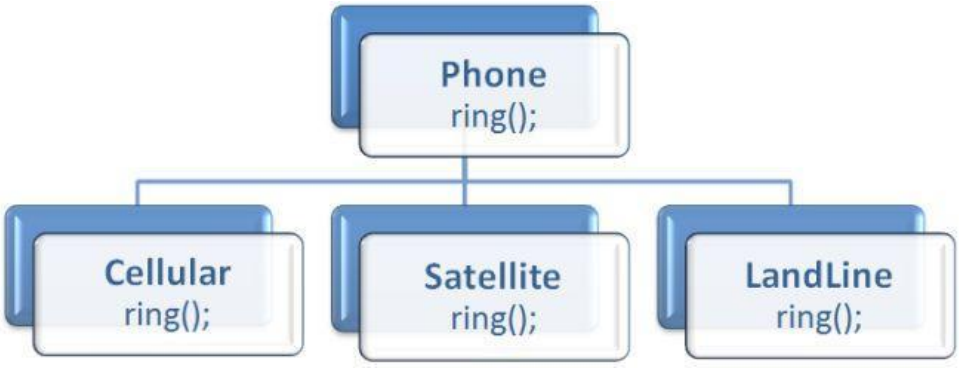
Method Overriding

Overriding is a feature that allows a subclass or child class to provide a specific implementation of a method that is already provided by one of its super-classes or parent classes. When a method in a subclass has the same name, same parameters or signature and same return type(or sub-type) as a method in its super-class, then the method in the subclass is said to override the method in the super-class. Method overriding is one of the ways by which C# achieve *Run Time Polymorphism(Dynamic Polymorphism)*.**Explanation**

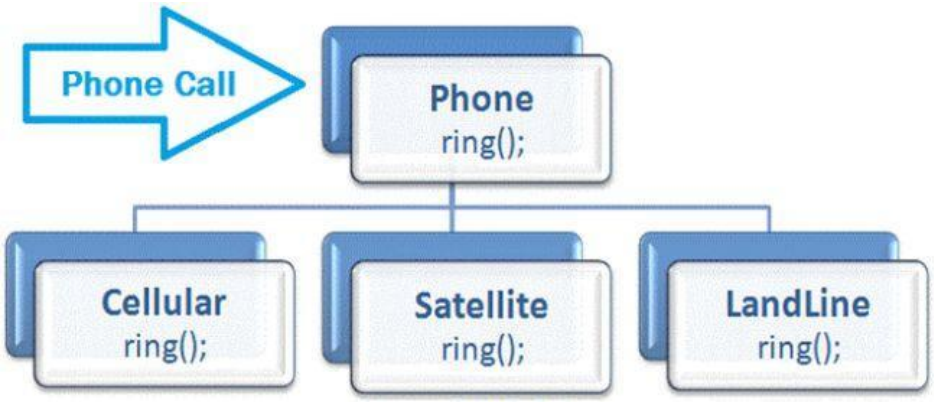
Suppose you have a Phone, no please don't suppose everyone has a phone today, although some people even have more than one. Okay leave it, so if you have phone:

Then there must be ring facility in it:

Now your phone can be of any type, like it can be cellular, satellite or landline, these all types of phones will also have the same or a different functionality (based on their attribute).

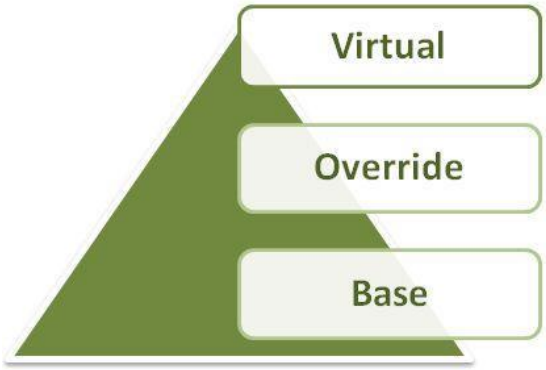


Now whenever you receive a call, the caller doesn't know whether you have a cellular phone, landline phone or anything else, he/she just calls you and according to that call operation your phone rings. In the case of method overriding, your phone works as a class and the ring is the functionality.



Keywords in Method Overriding

There are the following 3 types of keywords used in C# for method overriding:



Virtual Keyword

It tells the compiler that this method can be overridden by derived classes.

```
public virtual int myValue()
{
   -
   -
   -
}
```

Override Keyword

In the subclass, it tells the compiler that this method is overriding the same named method in the base class.

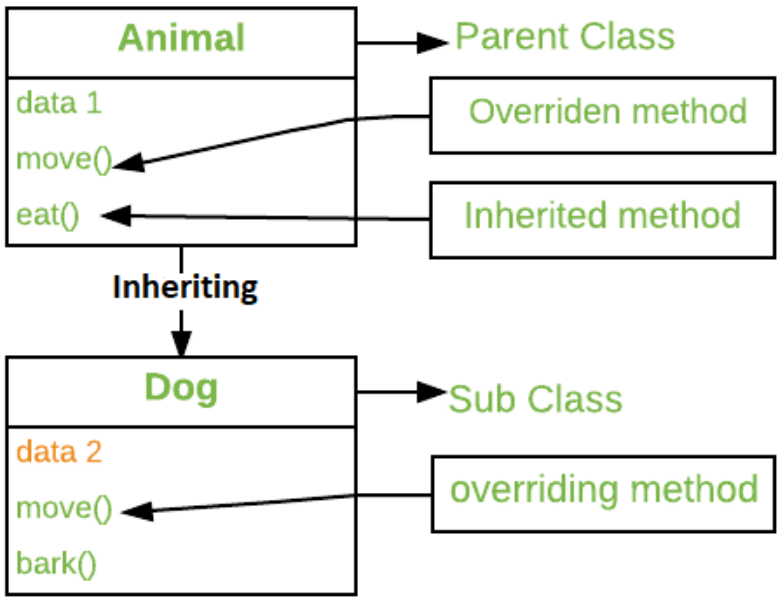
```
public override int myValue()
{
    -
    -
    -
}
```

Base Keyword

In the subclass, it calls the base class method for overriding functionality.

```
base.myValue();
```

The method that is overridden by an override declaration is called the overridden base method. An override method is a new implementation of a member that is inherited from a base class. The overridden base method must be virtual, abstract, or override.



class derived_class : base_class

{

```
    public void gfg();
}


class Main_Method
{
 static void Main()
 {
    derived_class d = new derived_class();
    d.gfg();
 }
}
```

Here the base class is inherited in the derived class and the method *gfg()* which has the same signature in both the classes, is overridden.

In C# we can use 3 types of keywords for Method Overriding:

- **virtual keyword:** This modifier or keyword use within base class method. It is used to modify a method in *base class* for *overridden* that particular method in the derived class.
- **override:** This modifier or keyword use with derived class method. It is used to modify a *virtual* or *abstract* method into *derived class* which presents in base class.

```
class base_class
{
    public virtual void gfg();
}


class derived_class : base_class
{
```

```
    public override void gfg();
}
class Main_Method
{
 static void Main()
 {
    derived_class d = new derived_class();
    d.gfg();

      base_class b = new derived_class();
       b.gfg();
 }
}
```

Here first, *d* refers to the object of the class *derived_class* and it invokes *gfg()* of the class *derived_class* then, *b* refers to the reference of the class base and it hold the object of class derived and it invokes *gfg()* of the class derived. Here *gfg()* method takes permission from base class to overriding the method in derived class.

Method Overriding without using virtual and override modifiers

```
// C# program to demonstrate the method overriding

// without using 'virtual' and 'override' modifiers

using System;
```

```
 // base class name 'baseClass'

class baseClass

 {

   public void show()

   {

      Console.WriteLine("Base class");

   }

}

 // derived class name 'derived'

// 'baseClass' inherit here

class derived : baseClass

{

       // overriding

   new public void show()

   {

      Console.WriteLine("Derived class");
```

```
    }

}

class GFG {

    // Main Method

    public static void Main()

    {

        // 'obj' is the object of

        // class 'baseClass'

        baseClass obj = new baseClass();

         // invokes the method 'show()'

        // of class 'baseClass'

        obj.show();

          obj = new derived();

         // it will invokes the method

        // 'show()' of class 'baseClass'

        obj.show();
```

```
  }

}
```

Output:

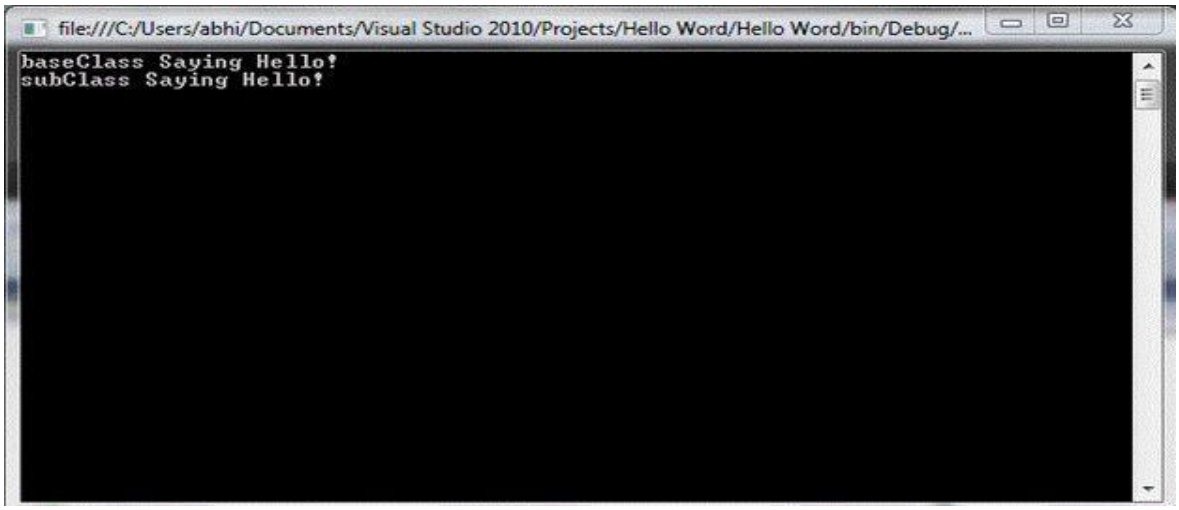
Base class

Base class

Method Overriding Example-2

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
namespace Hello_Word
{
  class baseClass
  {
    public virtual void Greetings()
    {
      Console.WriteLine("baseClass Saying Hello!");
    }
  }
  class subClass : baseClass
  {
    public override void Greetings()
    {
      base.Greetings();
      Console.WriteLine("subClass Saying Hello!");
    }
  }
  class Program
  {
```

```
    static void Main(string[] args)
    {
        baseClass obj1 = new subClass();
        obj1.Greetings();
        Console.ReadLine();
    }
  }
}
```



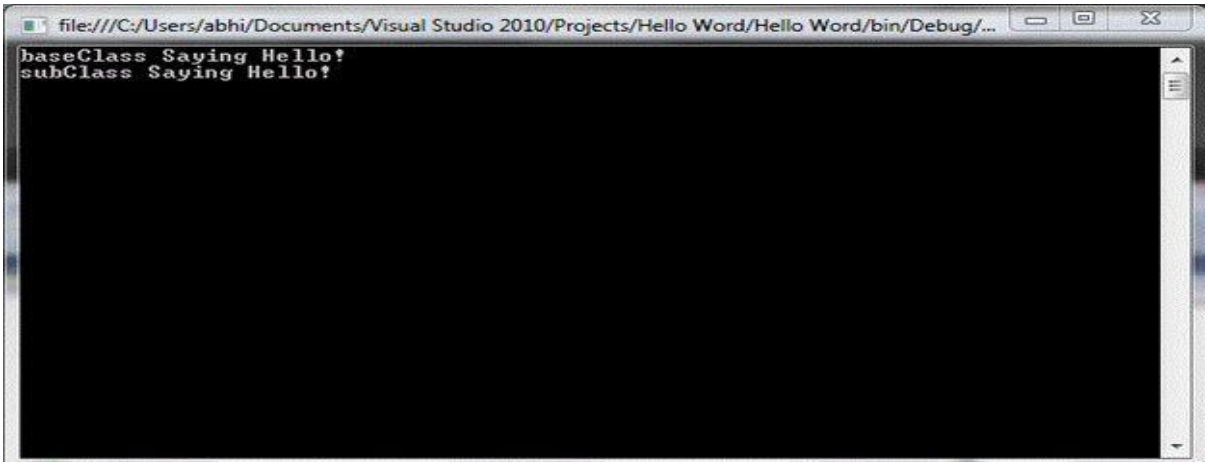
An output window is showing that baseClass will get called first and then subclass. If we make certain changes in this code like:

```
base.Greetings();
Console.WriteLine("subClass Saying Hello!"); // current scenario
```

After making a few modifications as in the following:

```
Console.WriteLine("subClass Saying Hello!");
base.Greetings(); // after modifications
```

Now our output will be like this:



If we remove "base.Greetings();" then the compiler will not call the baseclass as in the following:

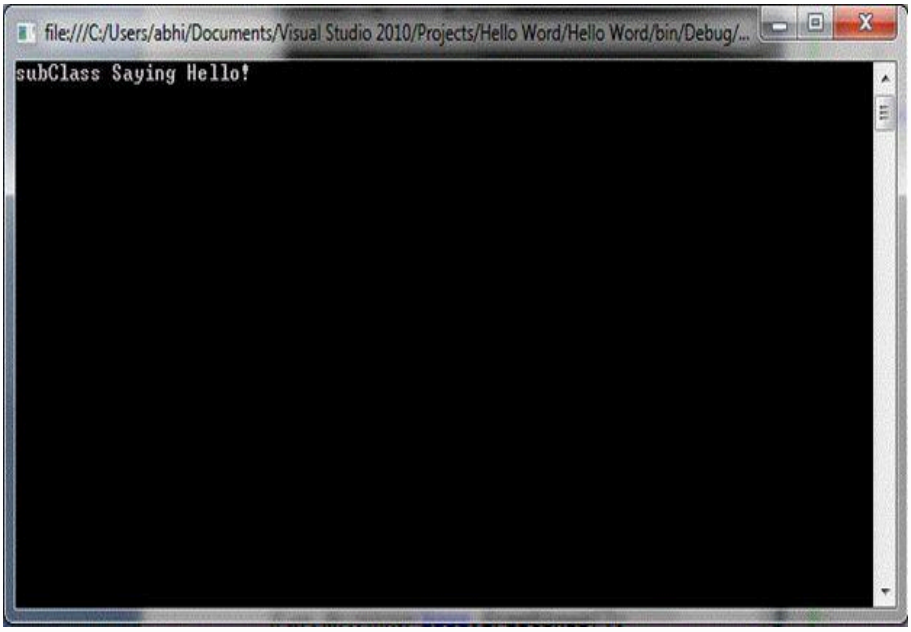
```
using System;

using System.Collections.Generic;

using System.Linq;
using System.Text;
namespace Hello_Word
{
   class baseClass
   {
      public virtual void Greetings()
      {
         Console.WriteLine("baseClass Saying Hello!");
      }
   }
   class subClass : baseClass
   {
      public override void Greetings()
      {
         Console.WriteLine("subClass Saying Hello!");
         // base.Greetings();
      }
   }
   class Program
```

```
  {
     static void Main(string[] args)
     {
        baseClass obj1 = new subClass();
        obj1.Greetings();
        Console.ReadLine();
     }
  }
```

OUTPUT



}

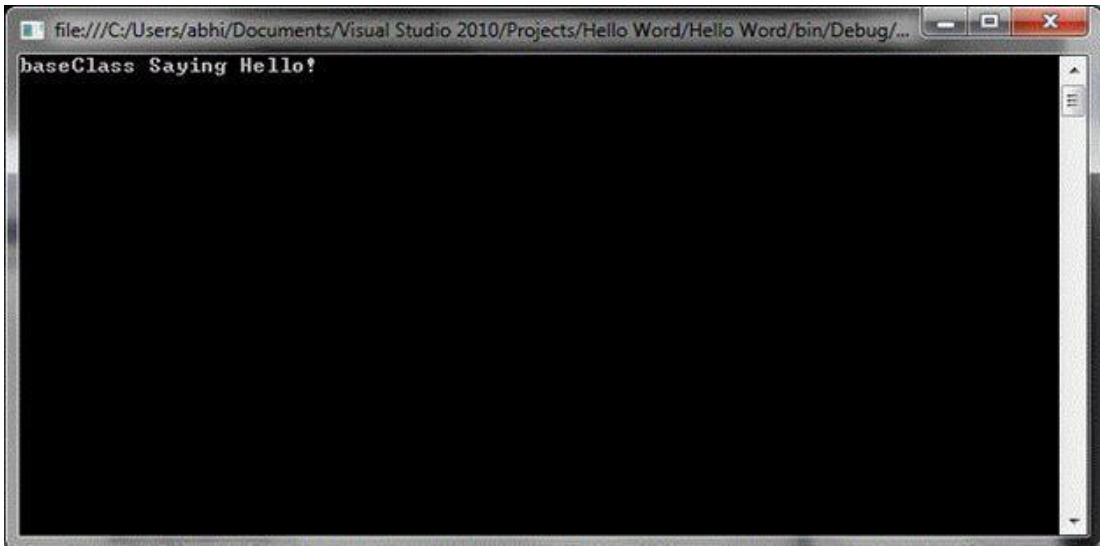
Now making some other changes, like placing:

```
baseClass obj1 = new baseClass();
obj1.Greetings();
```

Instead of:

```
baseClass obj1 = new subClass();
obj1.Greetings();
```

Now let’s check the output window, what’s its showing now:



So as you are seeing that now its calling only the baseClass. So now I guess you will be a little closer to the method overriding functionality.

C# Sealed

C# sealed keyword applies restrictions on the class and method. If you create a sealed class, it cannot be derived. If you create a sealed method, it cannot be overridden.

> ***Note: Structs are implicitly sealed therefore they can't be inherited.***

C# Sealed class

C# sealed class cannot be derived by any class. Let's see an example of sealed class in C#.

```
1. using System;
2. sealed public class Animal{
3.     public void eat() { Console.WriteLine("eating..."); }
4. }
5. public class Dog: Animal
6. {
7.     public void bark() { Console.WriteLine("barking..."); }
```

```
8. }
9. public class TestSealed
10. {
11.    public static void Main()
12.    {
13.       Dog d = new Dog();
14.       d.eat();
15.       d.bark();
16.
17.
18.    }
19. }
```

Output:

```
Compile Time Error: 'Dog': cannot derive from sealed type 'Animal'
```

C# Sealed method

The sealed method in C# cannot be overridden further. It must be used with override keyword in method.

Let's see an example of sealed method in C#.

```
1. using System;
2. public class Animal{
3.    public virtual void eat() { Console.WriteLine("eating..."); }
4.    public virtual void run() { Console.WriteLine("running..."); }
5.
6. }
7. public class Dog: Animal
8. {
9.    public override void eat() { Console.WriteLine("eating bread..."); }
10.   public sealed override void run() {
11.   Console.WriteLine("running very fast...");
12.   }
13. }
14. public class BabyDog : Dog
15. {
```

```
16.    public override void eat() { Console.WriteLine("eating biscuits..."); }
17.    public override void run() { Console.WriteLine("running slowly..."); }
18. }
19. public class TestSealed
20. {
21.    public static void Main()
22.    {
23.       BabyDog d = new BabyDog();
24.       d.eat();
25.       d.run();
26.    }
27. }
```

Output:

```
Compile Time Error: 'BabyDog.run()': cannot override inherited member 'Dog.run()' because it is sealed
```

Note: Local variables can't be sealed.

```
1.  using System;
2.  public class TestSealed
3.  {
4.     public static void Main()
5.     {
6.        sealed int x = 10;
7.        x++;
8.        Console.WriteLine(x);
9.     }
10. }
```

Output:

```
Compile Time Error: Invalid expression term 'sealed'
```

C# Sealed method

The sealed method in C# cannot be overridden further. It must be used with override keyword in method.

Let's see an example of sealed method in C#.

```
1. using System;
2. public class Animal{
3.     public virtual void eat() { Console.WriteLine("eating..."); }
4.     public virtual void run() { Console.WriteLine("running..."); }
5.
6. }
7. public class Dog: Animal
8. {
9.     public override void eat() { Console.WriteLine("eating bread..."); }
10.    public sealed override void run() {
11.    Console.WriteLine("running very fast...");
12.    }
13.}
14.public class BabyDog : Dog
15.{
16.    public override void eat() { Console.WriteLine("eating biscuits..."); }
17.    public override void run() { Console.WriteLine("running slowly..."); }
18.}
19.public class TestSealed
20.{
21.    public static void Main()
22.    {
23.        BabyDog d = new BabyDog();
24.        d.eat();
25.        d.run();
26.    }
27.}
```

Output:

```
Compile Time Error: 'BabyDog.run()': cannot override inherited member 'Dog.run()' because it is sealed
```

> ***Note: Local variables can't be sealed.***

```
1. using System;
2. public class TestSealed
3. {
4.     public static void Main()
5.     {
6.         sealed int x = 10;
7.         x++;
8.         Console.WriteLine(x);
9.     }
10.}
```

Output:

```
Compile Time Error: Invalid expression term 'sealed'
```

Why Sealed Classes?

- Sealed class is used to stop a class to be inherited. You cannot derive or extend any class from it.
- Sealed method is implemented so that no other class can overthrow it and implement its own method.
- The main purpose of the sealed class is to withdraw the inheritance attribute from the user so that they can't attain a class from a sealed class. Sealed classes are used best when you have a class with static members.
  e.g the "Pens" and "Brushes" classes of the System.Drawing namespace. The Pens class represents the pens for standard colors. This class has only static members. For example, "Pens.Red" represents a pen with red color.

Similarly, the “Brushes” class represents standard brushes. “Brushes.Red” represents a brush with red color.

C# abstract class and method

In C#, abstract class is a class which is declared abstract. It can have abstract and non-abstract methods. It cannot be instantiated. Its implementation must be provided by derived classes. Here, derived class is forced to provide the implementation of all the abstract methods.

Let's see an example of abstract class in C# which has one abstract method draw(). Its implementation is provided by derived classes: Rectangle and Circle. Both classes have different implementation.

In C#, we cannot create objects of an abstract class. We use the abstract keyword to create an abstract class. For example,

```
// create an abstract class
abstract class Language {
  // fields and methods
}
...

// try to create an object Language
// throws an error
Language obj = new Language();
```

An abstract class can have both abstract methods (method without body) and non-abstract methods (method with the body). For example,

```
abstract class Language {

  // abstract method
  public abstract void display1();

  // non-abstract method
```

```
  public void display2() {
    Console.WriteLine("Non abstract method");
  }
}
```

Before moving forward, make sure to know about C# inheritance.

C# Abstract class

```
1. using System;
2. public abstract class Shape
3. {
4.     public abstract void draw();
5. }
6. public class Rectangle : Shape
7. {
8.     public override void draw()
9.     {
10.        Console.WriteLine("drawing rectangle...");
11.    }
12.}
13.public class Circle : Shape
14.{
15.    public override void draw()
16.    {
17.        Console.WriteLine("drawing circle...");
18.    }
19.}
20.public class TestAbstract
21.{
22.    public static void Main()
23.    {
24.        Shape s;
25.        s = new Rectangle();
26.        s.draw();
```

```
27.        s = new Circle();
28.        s.draw();
29.    }
30. }
```

Output:

```
drawing ractangle...
drawing circle...
```

Inheriting Abstract Class

As we cannot create objects of an abstract class, we must create a derived class from it. So that we can access members of the abstract class using the object of the derived class. For example,

```
using System;
namespace AbstractClass {

  abstract class Language {

    // non-abstract method
    public void display() {
      Console.WriteLine("Non abstract method");
    }
  }

  // inheriting from abstract class
  class Program : Language {

    static void Main (string [] args) {

      // object of Program class
      Program obj = new Program();

      // access method of an abstract class
```

```
    obj.display();

    Console.ReadLine();
  }
 }
}
```

Output

```
Non abstract method
```

In the above example, we have created an abstract class named Language. The class contains a non-abstract method display().
We have created the Program class that inherits the abstract class. Notice the statement,

```
obj.display();
```

Here, obj is the object of the derived class Program. We are calling the method of the abstract class using the object obj.

> **Note**: We can use abstract class only as a base class. This is why abstract classes cannot be sealed. To know more, visit C# sealed class and method.

C# Abstract Method

A method that does not have a body is known as an abstract method. We use the abstract keyword to create abstract methods. For example,

```
public abstract void display();
```

Here, display() is an abstract method. An abstract method can only be present inside an abstract class.

When a non-abstract class inherits an abstract class, it should provide an implementation of the abstract methods.

Example: Implementation of the abstract method

```
using System;
namespace AbstractClass {

  abstract class Animal {

    // abstract method
    public abstract void makeSound();
  }

  // inheriting from abstract class
  class Dog : Animal {

    // provide implementation of abstract method
    public override void makeSound() {

      Console.WriteLine("Bark Bark");

    }

  }
  class Program  {
    static void Main (string [] args) {
      // create an object of Dog class
      Dog obj = new Dog();
      obj.makeSound();

      Console.ReadLine();
    }
  }
}
```

Output

```
Bark Bark
```

In the above example, we have created an abstract class named Animal. We have an abstract method makeSound() inside the class.
We have a Dog class that inherits from the Animal class. Dog class provides the implementation of the abstract method makeSound().

```
// provide implementation of abstract method
public override void makeSound() {

  Console.WriteLine("Bark Bark");

}
```

Notice, we have used override with the makeSound() method. This indicates the method is overriding the method from the base class.
We then used the object of the Dog class to access makeSound().
If the Dog class had not provided the implementation of the abstract method makeSound(), Dog class should have been marked abstract as well.

> **Note**: Unlike the C# inheritance, we cannot use virtual with the abstract methods of the base class. This is because an abstract class is implicitly virtual.

Abstract class with get and set accessors

We can mark get and set accessors as abstract. For example,

```
using System;
namespace AbstractClass {
  abstract class Animal {

    protected string name;
    // abstract method
    public abstract string Name {
      get;
      set;
    }
```

```
  }

  // inheriting from abstract class
  class Dog : Animal {

    // provide implementation of abstract method
    public override string Name {
      get {
        return name;
      }
      set {
        name = value;
      }
    }

  }
  class Program  {
    static void Main (string [] args) {
      // create an object of Dog class
      Dog obj = new Dog();
      obj.Name = "Tom";
      Console.WriteLine("Name: " + obj.Name);

      Console.ReadLine();
    }
  }
}
```

Output

```
Name: Tom
```

In the above example, we have marked the get and set accessor as abstract.

```
obj.Name = "Tom";
Console.WriteLine("Name: " + obj.Name);
```

We are setting and getting the value of the `name` field of the abstract class `Animal` using the object of the derived class `Dog`.

Access Constructor of Abstract Classes

An abstract class can have constructors as well. For example,

```
using System;
namespace AbstractClass {
  abstract class Animal {

    public Animal() {
      Console.WriteLine("Animal Constructor");
    }

  }

  class Dog : Animal {
    public Dog() {
      Console.WriteLine("Dog Constructor");
    }
  }

  class Program  {
    static void Main (string [] args) {
      // create an object of Dog class
      Dog d1 = new Dog();

      Console.ReadLine();
    }
  }
}
```

Output

```
Animal Constructor
Dog Constructor
```

In the above example, we have created a constructor inside the abstract class Animal.

```
Dog d1 = new Dog();
```

Here, when we create an object of the derived class Dog the constructor of the abstract class Animal gets called as well.

Interfaces in C#

Interfaces

Another way to achieve abstraction in C#, is with interfaces.

An interface is a completely "**abstract class**", which can only contain abstract methods and properties (with empty bodies):

Example

Syntax for Interface Declaration:

```
interface  <interface_name >
{
   // declare Events
   // declare indexers
   // declare methods
   // declare properties
}
```

```
// interface
interface Animal
{
```

```
  void animalSound(); // interface method (does not have a body)

  void run(); // interface method (does not have a body)

}
```

It is considered good practice to start with the letter "I" at the beginning of an interface, as it makes it easier for yourself and others to remember that it is an interface and not a class.

By default, members of an interface are abstract and public.

Note: Interfaces can contain properties and methods, but not fields.

To access the interface methods, the interface must be "implemented" (kinda like inherited) by another class. To implement an interface, use the : symbol (just like with inheritance). The body of the interface method is provided by the "implement" class. Note that you do not have to use the override keyword when implementing an interface:

The implementation of the interface's members will be given by class who implements the interface implicitly or explicitly.

- Interfaces specify what a class must do and not how.
- Interfaces can't have private members.
- By default all the members of Interface are public and abstract.
- The interface will always defined with the help of keyword '***interface***'.
- Interface cannot contain fields because they represent a particular implementation of data.
- *Multiple inheritance* is possible with the help of Interfaces but not with classes.
-

Example

```
// Interface

interface IAnimal

{
```

```
  void animalSound(); // interface method (does not have a body)
}
// Pig "implements" the IAnimal interface
class Pig : IAnimal
{
  public void animalSound()
  {
    // The body of animalSound() is provided here
    Console.WriteLine("The pig says: wee wee");
  }
}
class Program
{
  static void Main(string[] args)
  {
    Pig myPig = new Pig();  // Create a Pig object
    myPig.animalSound();
  }
}
```

OUTPUT

```
The pig says: wee wee
```

Notes on Interfaces:

- Like **abstract classes**, interfaces **cannot** be used to create objects (in the example above, it is not possible to create an "IAnimal" object in the Program class)
- Interface methods do not have a body - the body is provided by the "implement" class
- On implementation of an interface, you must override all of its methods
- Interfaces can contain properties and methods, but not fields/variables
- Interface members are by default abstract and public
- An interface cannot contain a constructor (as it cannot be used to create objects)

Why And When To Use Interfaces?

1) To achieve security - hide certain details and only show the important details of an object (interface).

2) C# does not support "multiple inheritance" (a class can only inherit from one base class). However, it can be achieved with interfaces, because the class can **implement** multiple interfaces. **Note:** To implement multiple interfaces, separate them with a comma

C# Interface example-2

Let's see the example of interface in C# which has draw() method. Its implementation is provided by two classes: Rectangle and Circle.

```
1.  using System;
2.  public interface Drawable
3.  {
4.     void draw();
5.  }
6.  public class Rectangle : Drawable
7.  {
8.     public void draw()
9.     {
10.       Console.WriteLine("drawing rectangle...");
```

```
11.    }
12. }
13. public class Circle : Drawable
14. {
15.    public void draw()
16.    {
17.       Console.WriteLine("drawing circle...");
18.    }
19. }
20. public class TestInterface
21. {
22.    public static void Main()
23.    {
24.       Drawable d;
25.       d = new Rectangle();
26.       d.draw();
27.       d = new Circle();
28.       d.draw();
29.    }
30. }
```

Output:

```
drawing ractangle...
drawing circle...
```

Note: Interface methods are public and abstract by default. You cannot explicitly use public and abstract keywords for an interface method.

Advantage of Interface:

- It is used to achieve loose coupling.
- It is used to achieve total abstraction.
- To achieve component-based programming
- To achieve multiple inheritance and abstraction.

- Interfaces add a plug and play like architecture into applications.

C# Exception Handling

Exception Handling in C# is *a process to handle runtime errors*. We perform exception handling so that normal flow of the application can be maintained even after runtime errors.

In C#, exception is an event or object which is thrown at runtime. All exceptions the derived from *System.Exception* class. It is a runtime error which can be handled. If we don't handle the exception, it prints exception message and terminates the program.

Advantage

It *maintains the normal flow* of the application. In such case, rest of the code is executed event after exception.

C# Exception Classes

All the exception classes in C# are derived from **System.Exception** class. Let's see the list of C# common exception classes.

Exception	Description
System.DivideByZeroException	handles the error generated by dividing a number with zero.
System.NullReferenceException	handles the error generated by referencing the null object.
System.InvalidCastException	handles the error generated by invalid typecasting.
System.IO.IOException	handles the Input Output errors.
System.FieldAccessException	handles the error generated by invalid private or protected field access.

C# Exception Handling Keywords

In C#, we use 4 keywords to perform exception handling:

- **Try , catch, finally, and throw**
-
  - **try −** A try block identifies a block of code for which particular exceptions is activated. It is followed by one or more catch blocks.
  - **catch −** A program catches an exception with an exception handler at the place in a program where you want to handle the problem. The catch keyword indicates the catching of an exception.
  - **finally −** The finally block is used to execute a given set of statements, whether an exception is thrown or not thrown. For example, if you open a file, it must be closed whether an exception is raised or not.
  - **throw −** A program throws an exception when a problem shows up. This is done using a throw keyword.

In C# programming, exception handling is performed by try/catch statement. The **try block** in C# is used to place the code that may throw exception. The **catch block** is used to handled the exception. The catch block must be preceded by try block.

Syntax for using try/catch looks like the following −

```
try {
  // statements causing exception
} catch( ExceptionName e1 ) {
  // error handling code
} catch( ExceptionName e2 ) {
  // error handling code
} catch( ExceptionName eN ) {
```

```
  // error handling code
} finally {
  // statements to be executed
}
```

C# example without try/catch

```
1. using System;
2. public class ExExample
3. {
4.     public static void Main(string[] args)
5.     {
6.         int a = 10;
7.         int b = 0;
8.         int x = a/b;
9.         Console.WriteLine("Rest of the code");
10.    }
11. }
```

Output: --- Unhandled Exception: System.DivideByZeroException: Attempted to divide by zero.

Example-2 C# try/catch

```
1. using System;
2. public class ExExample
3. {
4.     public static void Main(string[] args)
5.     {
6.         try
7.         {
8.             int a = 10;
9.             int b = 0;
```

```
10.         int x = a / b;
11.       }
12.       catch (Exception e) { Console.WriteLine(e); }
13.
14.       Console.WriteLine("Rest of the code");
15.    }
16. }
```

Output:

```
System.DivideByZeroException: Attempted to divide by zero.
Rest of the code
```

EXAMPLE-3

```
try
{
  int[] myNumbers = {1, 2, 3};
  Console.WriteLine(myNumbers[10]);
}
catch (Exception e)
{
  Console.WriteLine("Something went wrong.");
}
```

Output will be: - Something went wrong.

Automatic Memory Management

C# employs automatic memory management, which frees developers from manually allocating and freeing the memory occupied by objects. Automatic memory management policies are implemented by a ***garbage collector***. The memory management life cycle of an object is as follows.

1. When the object is created, memory is allocated for it, the constructor is run, and the object is considered live.

2. If the object, or any part of it, cannot be accessed by any possible continuation of execution, other than the running of destructors, the object is considered no longer in use, and it becomes eligible for destruction. The C# compiler and the garbage collector may choose to analyze code to determine which references to an object may be used in the future. For instance, if a local variable that is in scope is the only existing reference to an object, but that local variable is never referred to in any possible continuation of execution from the current execution point in the procedure, the garbage collector may (but is not required to) treat the object as no longer in use.

3. Once the object is eligible for destruction, at some unspecified later time, the destructor (§10.12) (if any) for the object is run. Unless overridden by explicit calls, the destructor for the object is run once only.

4. Once the destructor for an object is run, if that object (or any part of it) cannot be accessed by any possible continuation of execution, including the running of destructors, the object is considered inaccessible and the object becomes eligible for collection.

5. Finally, at some time after the object becomes eligible for collection, the garbage collector frees the memory associated with that object.

The garbage collector maintains information about object usage and uses this information to make memory management decisions, such as where in memory to locate a newly created object, when to relocate an object, and when an object is no longer in use or inaccessible.

Like other languages that assume the existence of a garbage collector, C# is designed so that the garbage collector may implement a wide range of memory management policies. For instance, C# does not require that destructors be run or that objects be collected as soon as they are eligible or that destructors be run in any particular order or on any particular thread.

The behavior of the garbage collector can be controlled, to some degree, via static methods on the class System.GC. This class can be used to request a collection to occur, destructors to be run (or not run), and so forth.
Because the garbage collector is allowed wide latitude in deciding when to collect objects and run destructors, a conforming implementation may produce output that differs from that shown by the following code. The program creates an instance of class A and an instance of class B.

```
using System;

class A

{

        ~A() {

                Console.WriteLine("Destruct instance of A");
        }
}

class B
{
        object Ref;

        public B(object o) {
                Ref = o;
        }
        ~B() {
                Console.WriteLine("Destruct instance of B");
        }
}

class Test
{
        static void Main() {
```

```
                B b = new B(new A());
                b = null;
                GC.Collect();
                GC.WaitForPendingFinalizers();


        }

}
```

These objects become eligible for garbage collection when the variable b is assigned the value null because after this time it is impossible for any user-written code to access them. The output could be either the following

```
Destruct instance of A
Destruct instance of B
```

or the following

```
Destruct instance of B
Destruct instance of A
```

because the language imposes no constraints on the order in which objects are garbage collected.

In subtle cases, the distinction between "eligible for destruction" and "eligible for collection" can be important. For example, in the following code.

Input and Output (Directories, Files, and streams)

A **file** is a collection of data stored in a disk with a specific name and a directory path. When a file is opened for reading or writing, it becomes a **stream**.

The stream is basically the sequence of bytes passing through the communication path. There are two main streams: the **input stream** and the **output stream**. The **input stream** is used for reading data from file (read operation) and the **output stream** is used for writing into the file (write operation).

Directories in C#

A directory is a file system that stores file. Now our task is to create a directory in C#. We can create a directory by using the CreateDirectory() method of the Directory class. This method is used to create directories and subdirectories in a specified path.

IO in C#

In c#, System.IO is a namespace. It will contain standard IO (input/output) types such as classes, structures, enumerations, and delegates to perform read/write operations on different sources like file, memory, network, etc.

C# I/O Classes

The System.IO namespace has various classes that are used for performing numerous operations with files, such as creating and deleting files, reading from or writing to a file, closing a file etc.

The following table shows some commonly used non-abstract classes in the System.IO namespace −

Sr.No.	I/O Class & Description
1	**BinaryReader** Reads primitive data from a binary stream.
2	**BinaryWriter** Writes primitive data in binary format.
3	**BufferedStream** A temporary storage for a stream of bytes.
4	**Directory** Helps in manipulating a directory structure.
5	**DirectoryInfo** Used for performing operations on directories.
6	**DriveInfo** Provides information for the drives.
7	**File** Helps in manipulating files.
8	**FileInfo** Used for performing operations on files.
9	**FileStream** Used to read from and write to any location in a file.
10	**MemoryStream** Used for random access to streamed data stored in memory.
11	**Path** Performs operations on path information.

12	**StreamReader** Used for reading characters from a byte stream.
13	**StreamWriter** Is used for writing characters to a stream.
14	**StringReader** Is used for reading from a string buffer.
15	**StringWriter** Is used for writing into a string buffer.

How will you manage input output directories in C#?

Two stream can be formed from file one is input stream which is used to read the file and another is output stream is used to write in the file. In C#, System.IO namespace contains classes which handle input and output streams and provide information about file and directory structure.

Examples

The following example shows **how to retrieve all the text files from a directory and move them to a new directory**. After the files are moved, they no longer exist in the original directory.

```
using System;
using System.IO;

namespace ConsoleApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            string sourceDirectory = @"C:\current";
            string archiveDirectory = @"C:\archive";
```

```
            try
            {
                var txtFiles =
Directory.EnumerateFiles(sourceDirectory, "*.txt");

                foreach (string currentFile in txtFiles)
                {
                    string fileName =
currentFile.Substring(sourceDirectory.Length + 1);
                    Directory.Move(currentFile,
Path.Combine(archiveDirectory, fileName));
                }
            }
            catch (Exception e)
            {
                Console.WriteLine(e.Message);
            }
        }
    }
}
```

The following example demonstrates how to use the EnumerateFiles method to retrieve a collection of text files from a directory, and then use that collection in a query to find all the lines that contain "Example".

The FileStream Class

The **FileStream** class in the System.IO namespace helps in reading from, writing to and closing files. This class derives from the abstract class Stream.

You need to create a **FileStream** object to create a new file or open an existing file. The syntax for creating a **FileStream** object is as follows −

```
FileStream <object_name> = new FileStream( <file_name>, <FileMode Enumerator>,
   <FileAccess Enumerator>, <FileShare Enumerator>);
```

For example, we create a FileStream object **F** for reading a file named **sample.txt as shown** −

```
FileStream F = new FileStream("sample.txt", FileMode.Open,
FileAccess.Read,
   FileShare.Read);
```

Sr.No.	Parameter & Description
1	**FileMode** Enumerator **The FileMode enumerator** defines various methods for opening files. The members of the FileMode enumerator are − ● **Append** − It opens an existing file and puts cursor at the end of file, or creates the file, if the file does not exist. ● **Create** − It creates a new file. ● **CreateNew** − It specifies to the operating system, that it should create a new file. ● **Open** − It opens an existing file. ● **OpenOrCreate** − It specifies to the operating system that it should open a file if it exists, otherwise it should create a new file. ● **Truncate** − It opens an existing file and truncates its size to zero bytes.
2	**FileAccess** **FileAccess** enumerators have members: **Read**, **ReadWrite** and **Write**.
3	**FileShare** **FileShare** enumerators have the following members − ● **Inheritable** − It allows a file handle to pass inheritance to the child processes ● **None** − It declines sharing of the current file ● **Read** − It allows opening the file for readin.

	• **ReadWrite** − It allows opening the file for reading and writing • **Write** − It allows opening the file for writing

Example

The following program demonstrates use of the **FileStream** class −

```
using System;
using System.IO;

namespace FileIOApplication {
  class Program {
    static void Main(string[] args) {
      FileStream F = new FileStream("test.dat", FileMode.OpenOrCreate,
        FileAccess.ReadWrite);

      for (int i = 1; i <= 20; i++) {
        F.WriteByte((byte)i);
      }
      F.Position = 0;
      for (int i = 0; i <= 20; i++) {
        Console.Write(F.ReadByte() + " ");
      }
      F.Close();
      Console.ReadKey();
    }
  }
}
```

OUTPUT

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 -1

FileMode Enumerator

The FileMode Enumerator defines methods for opening files. It is used to restrict the mode in which the file is opened by the user. The parameters to the enumerator check if the file is overwritten created or opened.

The members of the enumerator are as follows:

1) Append: It opens the file if it exists and places the cursor at the end of the file or creates a new file.

2) Create: It creates a new file

3) CreateNew: It specifies to an operating system that a new file should be created

4) Open: It opens an existing file

5) OpenOrCreate: It specifies to the system that open a file if it exists, else create a new file

6) Truncate: It opens an existing file. When opened, the file should be truncated, the size of the file is zero bytes

File Access Enumerator

The file is opened for both reading and writing operations. The enumerator is used to indicate whether user wants to read data from a file, write data to the file or perform both operations.

The members of the enumerator are Read, Write and ReadWrite.

FileShare Enumerator

The enumerator contains constants for controlling the access that the FileStream constructors have on the file. The use of the enumeration is to define whether two different applications can simultaneously read from the same file.

The members of the FileShare enumerator are as follows:

1) Inheritable: It allows a file handle to pass the inheritance to the child processes

2) None: It rejects sharing of the current file

3) Read: It allows the opening of a file for reading purpose

4) ReadWrite: It allows opening a file for the purpose of reading and writing

5) Write: It allows the opening of a file for writing

Advanced File Operations in C#

The preceding example provides simple file operations in C#. However, to utilize the immense powers of C# System.IO classes, you need to know the commonly used properties and methods of these classes.

Sr.No.	Topic & Description
1	**Reading from and Writing into Text files** It involves reading from and writing into text files. The **StreamReader** and **StreamWriter** class helps to accomplish it.
2	**Reading from and Writing into Binary files** It involves reading from and writing into binary files. The **BinaryReader** and **BinaryWriter** class helps to accomplish this.
3	**Manipulating the Windows file system**

	It gives a C# programamer the ability to browse and locate Windows files and directories.